

Link to Github: [2025-fpga-design-projects-fpgaall/README .md at main · hplp/2025-fpga-design-projects-fpgaall](https://github.com/2025-fpga-design-projects-fpgaall/README.md)

2. Project Overview and Design Exploration

In the early stages of our project, our primary goal was to implement an accurate lung cancer image classification system, with a focus on comparing software-based processing to hardware acceleration using an FPGA board. The overall objective was to determine which implementation—software or hardware—provides the most efficient balance between accuracy and processing speed for medical image classification.

We explored three main approaches during our design process:

Approach	Description	Outcome
Custom Verilog Design	Initially considered implementing the CNN from scratch in Verilog/VHDL	Rejected due to time constraints and complexity
DNNWeaver 2.0	Tried using the DNNWeaver framework to automatically generate Verilog from a trained model	Faced issues with Python environment compatibility and insufficient documentation
BNN-PYNQ (Final Choice)	Binary Neural Network overlays available for PYNQ-Z1 using Python & Jupyter	Chosen for rapid development, working overlays, and simplified hardware interfacing

To implement the final BNN-PYNQ approach, we followed several setup steps:

We cloned the official BNN-PYNQ GitHub repository.

We connected the PYNQ-Z1 board to our PC and retrieved its IP address to access it via a web browser.

We installed a fresh PYNQ v2.5 image on the SD card to ensure compatibility with the tutorial and overlays.

3. Architecture Selection and Data Organization

The objective in this phase was to validate and compare different neural networks available in the BNN-PYNQ tutorial, assessing their performance in terms of both accuracy and processing time.

After running the tutorial examples successfully on the board, we selected the best-performing model for our custom dataset. Our dataset was downloaded from Kaggle and required preprocessing before it could be used with the BNN overlay.

☒ Step 1: Confirm Directory Structure

Make sure your resized and cleaned dataset is here on the PYNQ:

```
/home/xilinx/jupyter_notebooks/lung_data_32x32/  
├─ train/  
│   ├── benign/  
│   └── malignant/  
├─ test/  
│   ├── benign/  
│   └── malignant/
```

If it's not named exactly like that, rename/move it for consistency.

☒ Step 2: Edit `cifar10.py` to Use Your Dataset

Go to:

```
cd ~/BNN-PYNQ/bnn/src/training/  
cp cifar10.py lung_cancer.py  
nano lung_cancer.py
```

The preprocessing included:

Writing Python scripts to reorganize the dataset into two labeled classes: **benign** and **malignant**.

Resizing all image data to compatible dimensions (32x32), to facilitate smoother integration with the FPGA overlay and ensure uniformity in training and inference.

This allowed us to move forward into customizing and deploying our classifier with real medical images.

4. Debugging and Implementation Challenges

During the hardware implementation, we faced a series of technical challenges while attempting to run the BNN-PYNQ overlays on the PYNQ-Z1 board:

Multiple Vivado Installations: We tested three different versions of Vivado (2021.2, 2019.1, and 2018.2). Only Vivado 2018.2 with HLS was fully compatible with the BNN-PYNQ toolchain.

Makefile Errors: The hardware build step (`make -hw`) consistently failed due to missing libraries and environment mismatches.

Missing Shared Object Files: Critical shared libraries such as `road-signs-cnv-pynq.so` and `libbnn.so` were not present after setup. Despite attempts to manually compile them using `make hw`, the required files were never generated.

Overlay Bitstream Issues: Custom overlays failed to expose required functions such as `load_parameters`, resulting in runtime errors.

Python Compatibility: Several Python versions were tested to find one that worked seamlessly with both PYNQ and the BNN codebase. Ultimately, Python 2.7 proved most stable.

We thoroughly debugged each of these issues by referencing community forums, official documentation, and related academic work. While some steps (like overlay loading and CFFI setup) were successful, key hardware functions remained non-functional due to the lack of valid `.so` libraries and IP block mismatches.

5. Reflection: Lessons Learned

What to Do:

- Use officially supported software versions (Vivado 2018.2, Python 2.7) for compatibility.
- Verify `.so` libraries and bitstreams post-installation.

- Always test with precompiled overlays before attempting customizations.
- Keep a clear record of each setup step for easier debugging.

What Not to Do:

- Avoid frequent switching between Vivado versions unless necessary.
- Do not assume all overlays include expected IP blocks.
- Do not rely on newer Python versions (e.g., 3.6+) unless proven compatible.
- Avoid skipping verification of libraries and binaries after cloning repositories.

6. Conclusion and Future Work

This project successfully developed a lung cancer classification system using a convolutional neural network in software, achieving over 90% classification accuracy on Kaggle medical image data. The dataset was carefully preprocessed, and a model was trained using PyTorch.

On the hardware side, we explored multiple tools including Verilog, DNNWeaver, and BNN-PYNQ within a V-model design approach. While software-based simulation and overlay setup were successful, the final hardware execution could not be completed due to persistent technical issues including Vivado compatibility, failed hardware compilation (make-hw), and missing .so libraries.

Despite our best efforts, we were unable to fully execute the classifier on the PYNQ-Z1 board. Nevertheless, this experience provided deep insight into FPGA workflows, toolchain dependencies, and practical challenges in deploying ML models to embedded systems.

Future Recommendations:

- Use Docker containers to preserve toolchain environments.
- Rebuild overlays from scratch ensuring IPs are correctly included.
- Consider using platforms with stronger BNN support, such as Ultra96.

This project serves as a strong foundational step toward future FPGA-accelerated AI systems.

V-Model Design Report: Lung Cancer Image Classification using FPGA (PYNQ Z1)

1. Introduction

This project applies a V-model lifecycle to the development of a lung cancer classification system using both software and FPGA-based hardware. It uses a dataset from Kaggle containing labeled lung images categorized as 'benign' or 'malignant'. A PyTorch CNN model is trained in software, and acceleration is explored on the PYNQ Z1 FPGA board. Due to limitations in model compatibility and board constraints, the CNN is not fully deployed on hardware. Instead, BNN-PYNQ is used as a feature extractor, and software classification is performed using the extracted features. Additional exploration using DNNWeaver was conducted for generating Verilog code for system-level synthesis.

2. V-Model Lifecycle

Left Arm: Specification & Design

Requirement Analysis

- Binary classification: benign vs malignant lung nodules.
- Dataset: Kaggle lung CT scan images resized to 64x64 pixels.
- Target: Software model accuracy >90%.
- Hardware implementation using PYNQ Z1 board.
- Compare classification performance (accuracy, latency, resource use) in software vs hardware.

System Design

- Part 1: PyTorch CNN trained on Kaggle dataset.
- Part 2: Feature extraction using CIFAR-10 model from BNN-PYNQ on PYNQ Z1.
- Classification on laptop using software model (due to BNN model limitations).

Architecture Design

- Software pipeline: CNN with Conv -> ReLU -> MaxPool -> FC layers.
- Hardware pipeline: BNN (binary conv + activation) on PYNQ extracts features.
- Use OpenCV for image preprocessing and resizing to 64x64.

- Communication: Extracted features sent from PYNQ to PC for classification.

Module Design

- CNN architecture includes:
 - 2 Conv layers (ReLU + MaxPool)
 - 1 FC hidden layer
 - 1 output layer with softmax
- Trained in PyTorch with train/val/test split
- BNN-PYNQ adapted to accept modified input size (64x64 images)
- Python scripts modified to preprocess input and extract features
- DNNWeaver explored with Vivado 2018.2 for HDL generation (but errors due to Python 2.7)

Right Arm: Implementation & Testing

Unit Implementation

- PyTorch CNN trained to >90% accuracy on validation data
- Hardware feature extraction verified using PYNQ + Jupyter interface
- BNN model remains unchanged (pretrained on CIFAR-10), used as generic extractor

Integration & Testing

- Process:
 - Image -> Preprocessed -> Sent to PYNQ
 - PYNQ returns feature vector -> Software classifier uses it
- Compared classification time with full software pipeline
- Observed lower latency in preprocessing with FPGA

System Verification

- Verified performance of hybrid model (hardware + software)
- FPGA usage constrained by memory (difficulty adapting CIFAR10 model)
- Software classification after feature extraction achieves similar accuracy (~90%)
- Classification time reduced for hardware preprocessing

Validation

- Validated predictions on test set with ground truth labels

- Limited testing on full hardware pipeline due to incompatibility of CNN structure
- Verified correctness of feature vector dimensions and output from PYNQ

3. Tools & Technologies

- PyTorch, OpenCV, NumPy, Jupyter
- Kaggle Lung Image Dataset (64x64, labeled)
- PYNQ Z1 board
- Vivado 2018.2
- DNNWeaver v2.0 (Verilog generation framework)
- BNN-PYNQ library
- Python 2.7 (required for PYNQ environment)

4. Challenges Faced

- BNN-PYNQ supports only CIFAR-10 model
- Incompatibility with custom lung dataset
- Python 2.7 limitation on PYNQ prevented usage of updated frameworks
- FINN installation failed (Docker versioning, dependencies)
- DNNWeaver raised Python errors in Jupyter environment
- Constrained memory on PYNQ Z1 limited full CNN deployment

5. Results & Evaluation

- Software CNN accuracy: ~91%
- Hardware feature extraction: Successful but limited to CIFAR-10 model
- Classification time: Faster with hardware preprocessing
- FPGA usage: Within resource limits using binary weights, no full CNN due to complexity

6. Conclusion

This project demonstrates a hybrid classification architecture that balances software accuracy and hardware acceleration. Although a full CNN deployment was not feasible on the PYNQ Z1, the BNN-PYNQ model effectively served as a feature extractor. Software classification maintained high accuracy. Exploration of DNNWeaver and FINN highlighted current toolchain limitations. The V-model lifecycle ensured a structured workflow from design to validation, making the approach suitable for embedded ML development.

7. Future Work

- Custom training of BNN with lung dataset
- Successful deployment of FINN with Docker
- Upgrade to more capable FPGA boards
- Use of quantization-aware training for better hardware mapping
- Deployment of lightweight CNN variants (e.g., MobileNet, TinyML models)